

Лабораторная работа №6. Flutter: StatefulWidget и управление состоянием

Цель работы: изучить разницу между `StatelessWidget` и `StatefulWidget`, научиться управлять состоянием приложения через `setState()`, подключать локальные изображения и обрабатывать нажатия кнопок — на примере слот-машины.

Часть 1. Подготовка проекта

Шаг 1. Создание проекта

1. Создайте новый проект:

```
flutter create slot_machine
```

```
cd slot_machine
```

```
mkdir img
```

2. Откройте проект в VS Code:

```
code .
```

3. Запустите приложение в Chrome и убедитесь что всё работает:

```
flutter run -d chrome
```

4. Нажмите **q** для выхода.

Шаг 2. Подключение assets

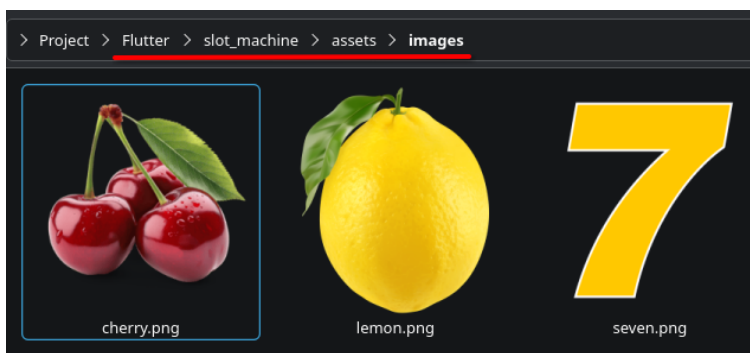
Картинки в Flutter не подключаются автоматически — их нужно зарегистрировать в `pubspec.yaml`. Это главный конфигурационный файл проекта: он описывает название, версию, зависимости и ресурсы приложения.

Создайте папку для картинок

```
mkdir -p assets/images
```

Скопируйте картинки

1. Скачайте архив с картинками с сайта и поместите три файла в папку `assets/images/`:

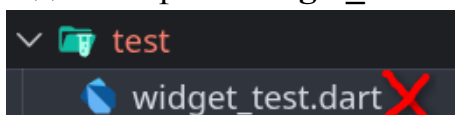


`assets/images/cherry.png` ← черешня

`assets/images/lemon.png` ← лимон

`assets/images/seven.png` ← семёрка

2. Удалите файл `widget_test.dart` в папке `test/`



Зарегистрируйте assets в pubspec.yaml

1. Откройте файл **pubspec.yaml** в корне проекта.
2. Почистите код, удалив все комментарии.
3. Найдите раздел **flutter:** и добавьте блок **assets:**

```
flutter:  
  uses-material-design: true  
  assets:  
    - assets/images/cherry.png  
    + - assets/images/lemon.png  
    - assets/images/seven.png
```

⚠ **pubspec.yaml** чувствителен к отступам — используйте только пробелы, не табуляцию. Два пробела на каждый уровень вложенности.

4. Сохраните файл. VS Code может предложить выполнить `flutter pub get` — согласитесь, либо выполните вручную в терминале:

flutter pub get

Эта команда читает **pubspec.yaml** и применяет все изменения — зависимости, шрифты, **assets**.

📸 **СКРИНШОТ 1:** Скриншот **pubspec.yaml** с добавленным блоком **assets** и папки **assets/images/** с тремя файлами в боковой панели VS Code. Сохраните в **img/step1_ВашаФамилия.png**

Шаг 3. Инициализация Git и подключение к GitHub

1. Инициализируйте репозиторий:

git init -b main

2. Откройте **.gitignore** и добавьте в конец:

ios/

linux/

macos/

windows/

.idea/

***.iml**

.DS_Store

Thumbs.db

3. Первый коммит:

git add .

git commit -m "Initial commit: create Flutter project with assets"

4. Создайте публичный репозиторий **Flutter_Lab5** на GitHub (без README, .gitignore и License) и привяжите:

git remote add origin <URL_вашего_репозитория>

git push -u origin main

 **СКРИНШОТ 2:** Скриншот терминала с результатом git push. Сохраните в **img/step2_ВашаФамилия.png**

Часть 2. StatefulWidget — виджет с состоянием

Шаг 1. Почему StatelessWidget больше не хватает

До сих пор все наши виджеты были **StatelessWidget** — они отрисовываются один раз и не меняются. Это отлично работает для статичного UI: текст, картинки, фон.

Но слот-машина должна **реагировать на действия пользователя** — при нажатии кнопки картинки меняются, счётчик монет обновляется. Виджет должен помнить своё текущее состояние и уметь его менять.

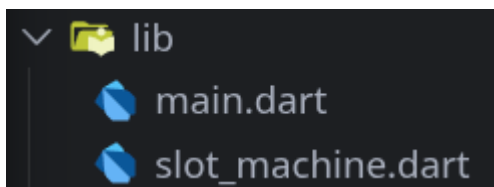
Для этого во Flutter есть **StatefulWidget**.

StatefulWidget состоит из двух классов:

- сам виджет — описывает конфигурацию
- объект **State** — хранит состояние и содержит метод **build()**

Шаг 2. Создаём SlotMachine

Создайте файл **lib/slot_machine.dart**. Будем строить класс постепенно.



Шаг 2.1 — Объявите **StatefulWidget**:

```
import 'package:flutter/material.dart';

class SlotMachine extends StatefulWidget {
  const SlotMachine({super.key});
}
```

Сразу появится ошибка:

```
SlotMachine extends StatefulWidget {  
  Missing concrete implementation of 'StatefulWidget.createState'.  
  Try implementing the missing method, or make the class  
  abstract. dart(non_abstract_class_inherits_abstract_member)
```

StatefulWidget требует реализовать метод **createState()** — он создаёт объект **State**, который будет управлять состоянием виджета.

Шаг 2.2 — Добавьте внутрь класса метод **createState()**:

```
class SlotMachine extends StatefulWidget {  
  const SlotMachine({super.key});  
  @override  
  State<SlotMachine> createState() => _SlotMachineState();  
}
```

SlotMachineState — это имя класса состояния. Подчёркивание в начале **** означает что класс **приватный** — он виден только внутри этого файла.

Шаг 2.3 — Создайте класс состояния (за пределами класса **SlotMachine**):

```
class _SlotMachineState extends State<SlotMachine> {  
  @override  
  Widget build(BuildContext context) {  
    return Container();  
  }  
}
```

Обратите внимание: **build()** теперь находится не в **StatefulWidget**, а в **State**. Именно **State** знает текущие данные и отвечает за отрисовку.

Шаг 3. Добавляем состояние и случайные символы

Теперь добавим данные которые будет хранить наш **State**.

Шаг 3.1 — Импортируйте **dart:math** для генерации случайных чисел:

```
import 'dart:math';
```

И объявите переменные состояния внутри класса **_SlotMachineState**.

1. **_random** — генератор случайных чисел из **dart:math**. **final** потому что сам генератор не меняется — меняются только числа которые он выдаёт:

```
final _random = Random();
```

2. **_symbols** — список путей к картинкам. По одному пути для каждого символа:

```
final _symbols = [
  'assets/images/cherry.png',
  'assets/images/lemon.png',
  'assets/images/seven.png',
];
```

3. **_coins** — текущий счёт монет, начинаем с 10.

_slot1, **_slot2**, **_slot3** — текущий символ в каждом из трёх барабанов.

_message — текст результата («Победа!» или «Попробуй ещё раз»):

```
var _coins = 10;
var _slot1 = 'assets/images/cherry.png';
var _slot2 = 'assets/images/lemon.png';
var _slot3 = 'assets/images/seven.png';
var _message = '';
```



Все переменные начинаются с **_** — они приватные для класса.

Шаг 4. Метод **_spin()** — логика броска

Добавьте метод **_spin()** внутри **_SlotMachineState** перед **build()**.

Эмодзи для вставки - 🙄, 🎉

```
void _spin() {
  setState(() {
    _slot1 = _symbols[_random.nextInt(_symbols.length)];
    _slot2 = _symbols[_random.nextInt(_symbols.length)];
    _slot3 = _symbols[_random.nextInt(_symbols.length)];

    if (_slot1 == _slot2 && _slot2 == _slot3) {
      _coins += 3;
      _message = 'Победа! 🎉 +3 монеты';
    } else {
      _coins -= 1;
      _message = 'Попробуй ещё раз 🙄 -1 монета';
    }
  });
}
```

_random.nextInt(_symbols.length) генерирует случайное число от 0 до 2 включительно — индекс случайного символа из списка.

Вся логика обернута в **setState()** — **Flutter** получит сигнал что состояние изменилось и вызовет **build()** заново с новыми значениями переменных.

Шаг 5. Строим UI

Теперь заполним метод **build()** реальным интерфейсом:

Эмодзи для вставки - 💰, 🎰

1. Меняем **Container** на **Column**:

```
return Column();
```

2. Делаем выравнивание по центру:

```
return Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
);
```

3. Добавляем текст с монетами:

```
return Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: [  
    Text(  
      '💰 Монеты: $_coins',  
      style: TextStyle(  
        fontSize: 28,  
        color: Colors.white,  
        fontWeight: FontWeight.bold,  
      ), // TextStyle  
    ), // Text  
  ],  
); // Column
```

4. Отступ:

```
      fontWeight: FontWeight.bold,  
    ), // TextStyle  
  ), // Text  
  + SizedBox(height: 40),  
],  
); // Column
```

5. Ряд со слотами:

```
    SizedBox(height: 40),  
    Row(  
      mainAxisAlignment: MainAxisAlignment.center,  
      children: [  
        Image.asset(_slot1, width: 100, height: 100),  
        SizedBox(width: 16),  
        + Image.asset(_slot2, width: 100, height: 100),  
        SizedBox(width: 16),  
        Image.asset(_slot3, width: 100, height: 100),  
      ],  
    ), // Row  
  ],
```

6. Сообщение:

```
), // Row
    SizedBox(height: 24),
    Text(
      _message,
      style: TextStyle(
        +   fontSize: 20,
        color: Colors.white,
      ), // TextStyle
    ), // Text
  ],
```

7. Кнопка:

```
    SizedBox(height: 40),
    ElevatedButton(
      onPressed: _spin,
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.amber,
        padding: EdgeInsets.symmetric(
          horizontal: 48,
          vertical: 16,
        ), // EdgeInsets.symmetric
      ),
      child: Text(
        +   'КРУТИТЬ 🎰',
        style: TextStyle(
          fontSize: 20,
          fontWeight: FontWeight.bold,
          color: Colors.black,
        ), // TextStyle
      ), // Text
    ), // ElevatedButton
  ],
```

Новые виджеты в этом шаге:

Виджет	Назначение
Column	Располагает дочерние виджеты вертикально
Row	Располагает дочерние виджеты горизонтально
mainAxisAlignment	Выравнивание вдоль основной оси (у Column — вертикальной, у Row — горизонтальной)
SizedBox	Пустое пространство заданного размера — отступ между виджетами
Image.asset()	Загружает картинку из папки assets
ElevatedButton	Кнопка с обработчиком onPressed

💡 **onPressed: _spin** — передаём метод как значение без скобок. Это значит: «вызови _spin когда кнопка нажата». Если написать `onPressed: _spin()` со скобками — метод вызовется сразу при построении UI, а не при нажатии.

Шаг 6. Подключаем SlotMachine в main.dart

Откройте **lib/main.dart**, удалите весь код и напишите. Импорты:

```
import 'package:flutter/material.dart';
import 'package:slot_machine/slot_machine.dart';
```

Метод **main()**:

```
void main() {
  runApp(
    MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        backgroundColor: Colors.deepPurple,
        body: SlotMachine(),
      ), // Scaffold
    ), // MaterialApp
  );
}
```

Запустите приложение и нажмите кнопку «КРУТИТЬ» — символы меняются случайно, монеты прибавляются при победе и убывают при проигрыше.

📸 **СКРИНШОТ 3:** Скриншот браузера с работающей слот-машиной. Сохраните в **img/step3_ВашаФамилия.png**

Выполните в терминале:

git add .

git commit -m "Add SlotMachine with StatefulWidget, random symbols and coin counter"

Часть 3. Улучшаем работу приложения

Шаг 1. Защита от отрицательных монет

Сейчас если монеты закончились — счётчик уходит в минус. Добавим проверку в метод **_spin()**: если монет не осталось, крутить нельзя.

Эмодзи для вставки - 😞

Добавьте в самое начало метода **_spin()**, до существующего **setState()**:

```
void _spin() {
  if (_coins <= 0) {
    setState(() {
      _message = 'Монеты закончились! 😞';
    });
    return;
  }

  setState(() {
```

💡 Обратите внимание на **return** после первого **setState()** — он прерывает выполнение метода. Это стандартный приём «ранний выход» (early return): сначала проверяем граничное условие, и если оно выполнено — выходим сразу, не выполняя основную логику.

Шаг 2. Блокируем кнопку когда монеты закончились

Сообщение об ошибке есть, но кнопка всё равно остаётся активной. Заблокируем её визуально — передадим **null** в **onPressed** когда монеты на нуле:

```
ElevatedButton(
  onPressed: _coins > 0 ? _spin : null,
```

_coins > 0 ? _spin : null — тернарный оператор, который вы знаете из Kotlin и C#. Если **onPressed** равен **null**, Flutter автоматически переводит кнопку в неактивное состояние и меняет её цвет.

Шаг 3. Кнопка «Сбросить»

Добавим кнопку сброса чтобы начать игру заново. Добавьте метод **_reset()** в класс **_SlotMachineState**:

```
void _reset() {
  setState(() {
    _coins = 10;
    _slot1 = 'assets/images/cherry.png';
    _slot2 = 'assets/images/lemon.png';
    _slot3 = 'assets/images/seven.png';
    _message = '';
  });
}
```

И добавьте кнопку в `build()` после кнопки «КРУТИТЬ»:

```
ElevatedButton( // ElevatedButton ...
  SizedBox(height: 12),
  TextButton(
    onPressed: _reset,
    child: Text(
      'Начать заново',
      style: TextStyle(
        color: Colors.white70,
        fontSize: 16,
      ), // TextStyle
    ), // Text
  ), // TextButton
),
```

TextButton — кнопка без фона, только текст. Используется для второстепенных действий чтобы не перегружать UI.

 **СКРИНШОТ 4:** Скриншот браузера с заблокированной кнопкой «КРУТИТЬ» при 0 монетах и активной кнопкой «Начать заново». Сохраните в **img/step4_ВашаФамилия.png**

Выполните в терминале:

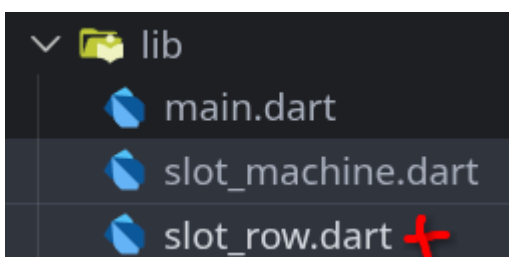
git add .

git commit -m "Add coin protection, disabled button state and reset button"

Шаг 4. Выносим виджет барабанов в отдельный класс

Посмотрите на метод `build()` — он уже достаточно большой. Вынесем три барабана в отдельный виджет **SlotRow**.

Создайте файл `lib/slot_row.dart`:



Напишите код:

```
import 'package:flutter/material.dart';

class SlotRow extends StatelessWidget {
  const SlotRow({
    super.key,
    required this.slot1,
    required this.slot2,
    required this.slot3,
  });

  final String slot1;
  final String slot2;
  final String slot3;

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Image.asset(slot1, width: 100, height: 100),
        SizedBox(width: 16),
        Image.asset(slot2, width: 100, height: 100),
        SizedBox(width: 16),
        Image.asset(slot3, width: 100, height: 100),
      ],
    ); // Row
  }
}
```

Обратите внимание на **required** перед параметрами конструктора. Это означает что при вызове виджета эти параметры **обязательны** — Flutter не позволит создать **SlotRow** без передачи всех трёх путей к картинкам. Это безопаснее чем опциональные параметры — ошибка будет поймана на этапе компиляции, а не во время работы приложения.

Обновите **slot_machine.dart** — добавьте импорт:

```
import 'package:slot_machine/slot_row.dart';
```

И замените **Row** в методе **build()** с картинками на **SlotRow**:

```
SlotRow(
  slot1: _slot1,
  slot2: _slot2,
  slot3: _slot3,
), // SlotRow
```

```
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Image.asset(_slot1, width: 100, height: 100),
    SizedBox(width: 16),
    Image.asset(_slot2, width: 100, height: 100),
    SizedBox(width: 16),
    Image.asset(_slot3, width: 100, height: 100),
  ],
), // Row
```

Выполните в терминале:

git add .

git commit -m "Extract SlotRow into a separate widget"

Часть 4. Реалистичная прокрутка барабанов

Сейчас слот-машина работает мгновенно — нажал кнопку, сразу получил результат. Это скучно. Настоящий игровой автомат крутит барабаны, замедляется и только потом показывает итог. Мы добавим именно это: анимацию вращения с тремя фазами скорости и поочерёдной остановкой барабанов.

Шаг 1. Добавляем флаг состояния `_isSpinning`

В классе `_SlotMachineState` рядом с остальными переменными добавляем одну новую:

```
var _message = '';  
var _isSpinning = false; +
```

Зачем: этот флаг будет равен `true` пока барабаны крутятся. Он нужен сразу в нескольких местах: чтобы заблокировать повторное нажатие кнопки, изменить её текст и показать анимацию мигания.

Шаг 2. Добавляем новый метод `_spinReel()`

Добавляем метод **перед** существующим `_spin`. Это вспомогательный метод — он отвечает за один барабан.

```
Future<String> _spinReel({  
  required int totalTicks,  
  required void Function(String) onTick,  
}) async {  
  String result = _symbols[0];  
  for (int i = 0; i < totalTicks; i++) {  
    // Три фазы скорости в зависимости от прогресса  
    final progress = i / totalTicks;  
    final delay = progress < 0.5  
      ? 40 // ● быстрая фаза — картинки мелькают  
      : progress < 0.8  
      ? 100 // ● замедление  
      : 200; // ● почти стоп — максимальная интрига  
    await Future.delayed(Duration(milliseconds: delay));  
    result = _symbols[_random.nextInt(_symbols.length)];  
    onTick(result);  
  }  
  return result;  
}
```

Разбор логики:

- **totalTicks** — сколько раз барабан сменит картинку. Разные барабаны получают разные значения (10, 13, 16), поэтому они остановятся по очереди.
- **progress** — число от 0.0 до 1.0, показывает «насколько далеко» прошёл барабан.
- Три порога скорости создают эффект замедления: сначала быстро, потом медленнее, потом почти стоп.
- **onTick** — колбэк, который вызывается каждый тик и обновляет картинку в UI через **setState**.

Шаг 3. Заменяем метод `_spin()`

Старый `_spin` был синхронным. Заменяем его **полностью** на асинхронную версию:

```
Future<void> _spin() async {
  if (_coins <= 0 || _isSpinning) return;
  setState(() {
    _isSpinning = true;
    _message = '';
  });
  // Барабаны крутятся разное количество тиков
  final result1 = await _spinReel(
    totalTicks: 10,
    onTick: (val) => setState(() => _slot1 = val),
  );
  final result2 = await _spinReel(
    totalTicks: 13,
    onTick: (val) => setState(() => _slot2 = val),
  );
  final result3 = await _spinReel(
    totalTicks: 16,
    onTick: (val) => setState(() => _slot3 = val),
  );
}
```

Продолжение на следующей странице (продолжаем писать внутри `_spin()`):

```
// Короткая пауза перед результатом — финальная интрига
await Future.delayed(Duration(milliseconds: 300));
setState(() {
  _isSpinning = false;
  if (result1 == result2 && result2 == result3) {
    if (result1 == 'assets/images/seven.png') {
      _coins += 10;
      _message = 'ДЖЕКПОТ! 🎰🎰🎰 +10 монет';
    } else {
      _coins += 3;
      _message = 'Победа! 🍒 +3 монеты';
    }
  } else {
    _coins -= 1;
    _message = 'Попробуй ещё раз 😞 -1 монета';
  }
});
}
```

Что изменилось и почему:

Старый код	Новый код	Причина
void _spin()	Future<void> _spin() async	Нужны await внутри
Проверка только _coins <= 0	Добавлена проверка _isSpinning	Блокирует повторный запуск
Результат сразу случайный	Берём из _spinReel	Это то, на чём остановился барабан
Только победа +3	Добавлен джекпот на семёрках +10	Новая механика

Шаг 4. Обновляем метод _reset()

Добавляем сброс нового флага. Находим строку `_message = ''`; в методе `_reset()` и добавляем после неё:

```
void _reset() {
  setState(() {
    _coins = 10;
    _slot1 = 'assets/images/cherry.png';
    _slot2 = 'assets/images/lemon.png';
    _slot3 = 'assets/images/seven.png';
    _message = '';
    +_isSpinning = false; // ← добавляем
  });
}
```

Зачем: если вдруг сбросить игру во время вращения, флаг должен вернуться в false, иначе кнопка останется заблокированной навсегда.

Шаг 5. Обновляем виджеты в методе build

Здесь мы **не переписываем весь build** — только находим конкретные виджеты и заменяем их.

5.1. SlotRow — добавляем мигание

Находим в **build** виджет **SlotRow(...)** и оборачиваем его в **AnimatedOpacity**:

Было:

```
SlotRow(  
  slot1: _slot1,  
  slot2: _slot2,  
  slot3: _slot3,  
) , // SlotRow
```

Стало:

```
AnimatedOpacity(  
  opacity: _isSpinning ? 0.85 : 1.0,  
  duration: Duration(milliseconds: 100),  
  child: SlotRow(  
    slot1: _slot1,  
    slot2: _slot2,  
    slot3: _slot3,  
  ) , // SlotRow  
) , // AnimatedOpacity
```

Зачем: небольшое мигание визуально подчёркивает, что барабаны в движении.

AnimatedOpacity плавно анимирует переход прозрачности.

5.2. Текст сообщения — добавляем анимированную смену

Находим **Text(_message, ...)** и заменяем его на:

Было:

```
Text(  
  _message,  
  style: TextStyle(  
    fontSize: 20,  
    color: Colors.white,  
  ) , // TextStyle  
) , // Text
```

Стало:

```

SizedBox(
  height: 36,
  child: AnimatedSwitcher(
    duration: Duration(milliseconds: 400),
    child: Text(
      _isSpinning ? '🎰 Крутим...' : _message,
      key: ValueKey(
        _isSpinning ? 'spinning' : _message,
      ), // ValueKey
      style: TextStyle(
        fontSize: 20,
        color: Colors.white,
        fontWeight: _message.contains('ДЖЕКПОТ')
          ? FontWeight.bold
          : FontWeight.normal,
      ), // TextStyle
    ), // Text
  ), // AnimatedSwitcher
), // SizedBox

```

Зачем: **AnimatedSwitcher** анимирует смену текста. **ValueKey** обязателен — без него Flutter не поймёт, что текст изменился и анимация не сработает. **SizedBox** фиксирует высоту, чтобы кнопка не прыгала при смене текста.

5.3. Кнопка «Крутить» — блокируем во время вращения

Находим **ElevatedButton(...)** и заменяем **onPressed** и текст внутри:

Было:

```

ElevatedButton(
  onPressed: _coins > 0 ? _spin : null,
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.amber,
    padding: EdgeInsets.symmetric(
      horizontal: 48,
      vertical: 16,
    ), // EdgeInsets.symmetric
  ),
  child: Text(
    'Крутить 🎰',
  ),
)

```

Стало:

```
ElevatedButton(  
  onPressed: _coins > 0 && !_isSpinning  
    ? _spin  
    : null,  
  style: ElevatedButton.styleFrom(  
    backgroundColor: Colors.amber,  
    padding: EdgeInsets.symmetric(  
      horizontal: 48,  
      vertical: 16,  
    ), // EdgeInsets.symmetric  
  ),  
  child: Text(  
    _isSpinning ? '🎰 Крутим...' : 'КРУТИТЬ 🎰',  
    style: TextStyle(  

```

Зачем: добавляем второе условие `&& !_isSpinning`. Когда кнопка получает `null` вместо функции — Flutter автоматически делает её серой и неактивной. Это стандартный паттерн в Flutter.

5.4. Кнопка «Начать заново» — блокируем во время вращения

Находим `TextButton(onPressed: _reset, ...)` и меняем `onPressed`:

Было:

```
TextButton(  
  onPressed: _reset,  
  child: Text(  

```

Стало:

```
TextButton(  
  onPressed: _isSpinning ? null : _reset,  
  child: Text(  

```

Зачем: нельзя сбрасывать игру пока барабаны крутятся — это прервёт `async` операцию и приведёт к непредсказуемому состоянию.

Итог

После всех изменений слот-машина:

1. Крутит барабаны с тремя фазами — быстро → медленнее → почти стоп
2. Останавливает барабаны по очереди — первый, потом второй, потом третий
3. Блокирует UI во время вращения — нельзя нажать кнопку дважды

4. Показывает визуальную обратную связь — мигание, анимированный текст, изменение надписи на кнопке

5. Добавляет джекпот — три семёрки дают +10 монет вместо +3

Поэкспериментируйте

Попробуйте изменить цифры и посмотреть что получится:

- 40 / 100 / 200 → скорость фаз. Что если сделать 20 / 60 / 300?
- 10 / 13 / 16 → количество тиков барабанов. Что если все три одинаковые?
- 300 → пауза перед результатом. Попробуйте 800 — почувствуете разницу
- 0.5 / 0.8 → границы фаз. Что если сделать 0.3 / 0.6?

Коммит

git add .

git commit -m "Bonus: realistic slot animation with easing and AnimatedOpacity"

git push

Самостоятельные задания

Задание 1. Практика

Создать новый проект с нуля и написать приложение-визитка: имя студента, группа, любимый язык программирования. Три отдельных виджета в трёх файлах, градиентный фон, каждый Text стилизован. Обязательно коммиты по ходу работы, не один в конце.

Задание 2. README.md

Оформите **README.md** в репозитории **Flutter_Lab6**:

- Название лабораторной и номер
- Фамилия, имя, группа, дата сдачи
- Что изучили (3–5 пунктов своими словами)
- Скриншот финального приложения
- **Инструкция по запуску**

Выполните в терминале:

git add .

git commit -m "added README description and completed labs"

git push